
Recitation 4: MongoDB

Why use a database?

A web app needs state to survive beyond one browser tab, one server process, or one page refresh.

1. **Persistence** → data still exists after a refresh, deploy, crash, or cache clear
2. **Concurrent changes** → multiple users can create, modify, and delete share data
3. **Searchability** → the backend can find the right data and return only what the UI actually needs

Two ways to organize the same app data

Relational database → data as tables and relationships

movies

id	title	year
m1	Inception	2010
m2	Interstellar	2014

genres

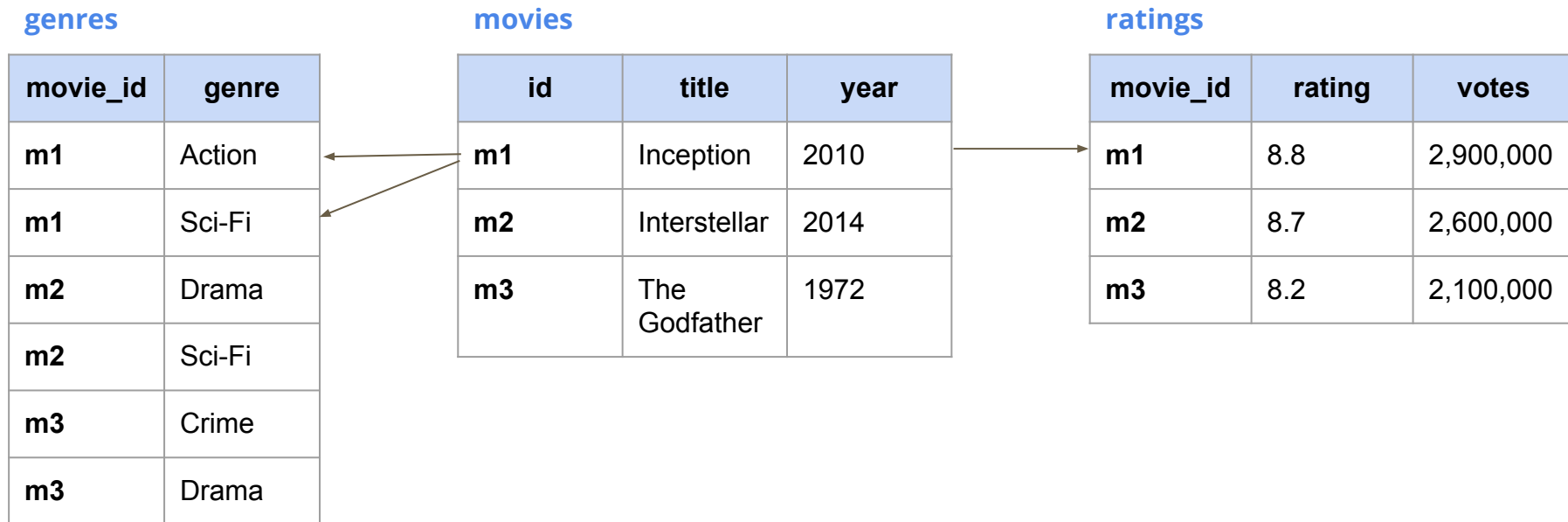
movie_id	genre
m1	Action
m1	Sci-Fi
m2	Drama

Document database → data as documents that can nest arrays/objects

```
{
  _id: "m1",
  title: "Inception",
  year: 2010,
  genres: ["Action", "Sci-Fi"],
  imdb: { rating: 8.8, votes: 2900000 }
}
```

Main contrast: split data across relations vs. keep related fields in one tree

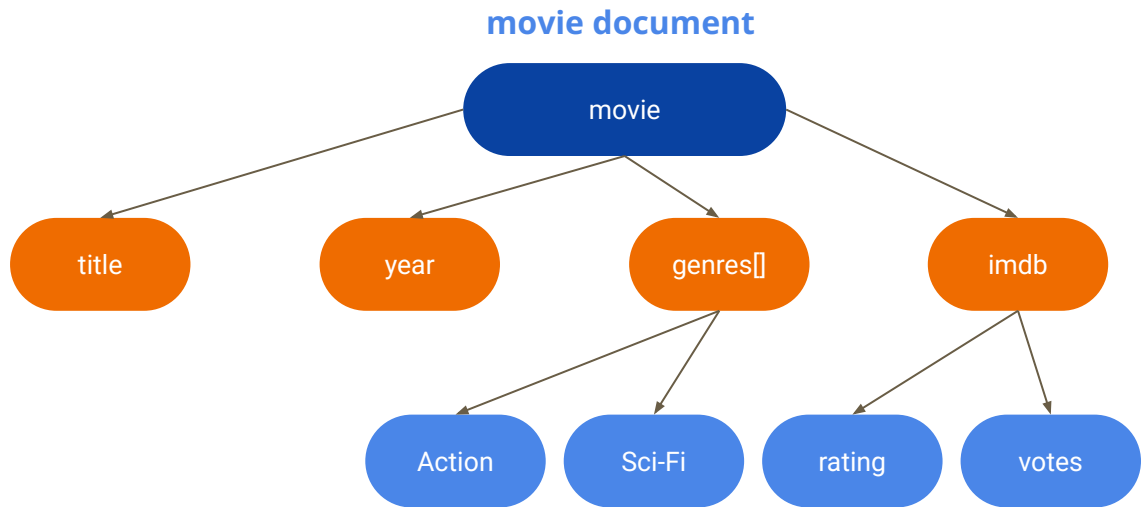
Relations make connections explicit



Main idea: to reconstruct one movie, the application may need to combine rows from multiple tables

MongoDB documents are tree-shaped

```
{  
  _id: "m1",  
  title: "Inception",  
  year: 2010,  
  genres: ["Action", "Sci-Fi"],  
  imdb: { rating: 8.8, votes: 2900000 }  
}
```



A MongoDB filter can follow these branches using dot notation.

What are the benefits/drawbacks of one over the other?

Common MongoDB vocabulary

- **Cluster** → the hosted MongoDB deployment
- **Database** → a named group of related collections
- **Collection** → a named group of related documents
- **Document** → one stored object; unit of storage
- **Field** → a key/value pair inside a document

_id

Every document has a unique identifier. MongoDB can generate an ObjectId, or your code can set its own _id.

MongoDB filters are document-shaped predicates

```
const query = { title: "Inception" };  
const movie = await  
movies.findOne(query);
```

Common filter patterns

- **Literal value:** { year: 2014 }
- **Comparison:** { year: { \$gte: 2010 } }
- **Logical:** { \$or: [{ year: 2010 }, { year: 2014 }] }
- **Array contains:** { genres: "Sci-Fi" }
- **Nested field:** { "imdb.rating": { \$gte: 8 } }

Note

The shape of your filter mirrors the shape of your document

Find returns a cursor

```
const cursor = movies.find({
  genres: "Sci-Fi",
  "imdb.rating": { $gte: 8 }
});

const results = await cursor
  .sort({ year: -1 })
  .limit(10)
  .project({ title: 1, year: 1, imdb: 1 })
  .toArray();
```

Cursor methods

sort: order the result set

limit: cap the number of results

skip: pagination offset

project: choose fields to return

toArray: materialize results

Example endpoint: GET /movies?genre=Sci-Fi&minRating=8

Updates happen through backend operations

We have seen that concepts have actions and queries. But how are these actually implemented them?

The backend can handle the actions and queries through MongoDB operation(s)

open (registry: Registry)
where registry exists and is not active
then make registry active

```
await registries.updateOne(  
  { _id: registry, active: false },  
  { $set: { active: true } }  
);
```

_getOpenRegistries (): many (registry: Registry)
return all active registries

```
const openRegistries = await registries  
  .find({ active: true })  
  .toArray();
```

Activity:

Your team will fill one slide.

Pick a reasonable MongoDB representation for the feature you are given

Deliverable

- Collections + example document(s)
- One query or update
- The backend endpoint that would run it

Discussion

- What app action is this supporting
- What query becomes easy
- What update becomes awkward
- How might this scale or break?

Example: Gift registry

Prompt

Model a gift registry

A recipient creates a registry with requested items. Givers should see what is still available and mark an item as purchased.

Model

Registries

```
{
  _id: "r1",
  recipientId: "u1",
  status: "open",
  items: [
    {
      itemId: "i1",
      name: "Coffee Maker",
      requested: 2,
      purchased: 1
    },
    {
      itemId: "i2",
      name: "Blender",
      requested: 1,
      purchased: 0
    }
  ]
}
```

Operation

A user purchases a coffee maker from registry r1

```
await registries.updateOne(
  { _id: "r1", "items.itemId": "i1" },
  { $inc: { "items.$.purchased": 1 } }
);
```

POST
/registries/r1/items/i1/purchase

1: Office hours queue

Prompt

Model questions in a shared OH queue

A student creates a question with a topic and text. A staff member can claim it. Later the question is marked resolved.

Model

Operation

2: Study group finder

[Your names here]

Prompt

Model study groups for classes.

Students can create groups with a course, topic, meeting time, max size, and members. The UI should show groups with open seats.

Model

Operation

3: Leaderboard

[Your names here]

Prompt

Model a leaderboard

Players can submit scores.
The UI should show the
top 10 scores and allow
each player to see their
own submissions

Model

Operation

4: Movie watchlist

Prompt

Model a movie watchlist

Users save movies, mark them watched, and optionally add a rating/review. The UI should show one user's unwatched movies.

Model

Operation

5: Restaurant reservation

Prompt

Model a restaurant reservation system

A restaurant has tables with different capacities. Diners can make reservations for a specific date/time, party size, table (bar vs. indoor seating, vs. outdoor seating), and can later view or cancel. A restaurant can confirm the reservation.

Model

Operation

6: Class waitlist

[Your names here]

Prompt

Model a class waitlist.

The students puts themselves in the database. The registrar has to add/remove students from the queue, remove/add to the class, weight students in the queue based on certain attributes (major, year of graduation, etc).

Model

Operation

Reminders

- class website @ web.mit.edu/6.104
- [assignment P1](#) due Monday 11:59pm
- [office hours](#) in 34-304 - Sunday 1-3pm, Monday 6-8pm, Friday 3-5pm
- s^3 email list going forward (CC your s^3 dean): 6.104-s3@mit.edu
- post any questions/concerns on Commons (do not email staff)